College of Engineering and Information Technology

Flying Radar: A Drone-Based Software-Defined Radar

Commercial In Confidence

ENGI 4900: Final Report

13 Nov 2026

36/36 Pages

Summary of AI Use

Categories of artificial intelligence (AI) tool use in this assignment are as indicated in the following table.

AI tool use category	**Used**
Spelling	✓
Grammar	✓
Other copy editing	
Literature review support	
Visuals	
Substantive editing	
Idea generation	
Translation	
Research technical support	✓
No AI tool was used	

Acknowledgements

Contents

Summary of AI Use **i**

Acknowledgements **ii**

List of Figures **v**

List of Tables **vi**

List of Acronyms **viii**

Contribution Statement **ix**

4 Methods, Results and Discussion **xi**

5 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements **xii**

5.1 Introduction xii

5.2 Flight Controller Configuration xii

5.3 Final System xiii

5.3.1 MAVLink Interface xiii

5.3.2 Test: RC Switch Trigger Latency xiv

5.3.3 Telemetry Log xv

5.3.4 Test: GPS Drift xv

5.3.5 SDR Interface xvi

5.3.6 Test: Computational Overhead xvii

5.3.7 Post-Processing xvii

5.4 Summary xviii

References **xviii**

Statement of AI Use **xix**

6 System Design **xx**

6.1 System Architecture xx

6.2 Subsystem Interfaces xx

A Objective 4: Supporting Material **xxi**

A.1 SSH Setup xxi

A.2 GPIO Packages Install xxi

A.3 Virtual Environment and `rpi-lgpio` Install xxi

A.4 GPIO Read Script xxi
A.5 Telemetry Data Visualisation Script xxii

List of Figures

5.1 Activation signal flow starting at the pilot transmitter and ending at the software-defined radar (SDR). xii

5.2 Captured screenshot of live data display for the MAVLink interface (Note: data has demo values). xv

5.3 State diagram of radar control sequence xvi

5.4 Flight path and radar capture locations plotted in the data visualisation script. . xviii

6.1 High level diagram of drone-radar integration system architecture xx

List of Tables

5.1 MAVLink messages requested from the flight controller and their use. xiii
5.2 Trigger-to-capture latency after removing the 4 s settle-delay from logs 1–4 . . . xiv
5.3 Mean position of each stationary window xv
5.4 Horizontal GPS drift and satellites visible for each stationary window xvi
5.5 Raspberry Pi resource usage for each test condition xvii

6.1 Interfaces between subsystems xx

List of Acronyms

Acronym	Definition
3D	three dimensional
AC	alternating current
AI	artificial intelligence
API	application programming interface
ASA	acrylonitrile styrene acrylate
AST	Advanced Systems & Technologies
CLI	command-line interface
CPU	central processing unit
C-UAS	counter-uncrewed aerial system
CAD	computer-aided design
COTS	commercial off-the-shelf
CW	continuous wave
DC	direct current
DEM	digital elevation model
FMCW	frequency modulated continuous wave
GPIO	general purpose input/output
GPR	ground penetrating radar
GPS	global positioning system
HALE	high-altitude long-endurance
HTML	Hypertext Markup Language
IDE	integrated development environment
IED	improvised explosive device
InSAR	interferometric synthetic aperture radar
IQ	in-phase and quadrature

JSON	JavaScript Object Notation
JST	Japan solderless terminal
LiPo	lithium-polymer
PC	personal computer
PCB	printed circuit board
PID	product identifier
PLA	polylactic acid
RAM	random-access memory
RF	radio frequency
SAR	synthetic aperture radar
SBC	single-board computer
SD	secure digital
SDR	software-defined radar
SI	International System of Units
SLD	single-line diagram
SSH	secure shell
UAV	uncrewed aerial vehicle
USB	universal serial bus
VID	vendor identifier

Contribution Statement

This contribution statement records the specific contribution of each team member to the technical work and preparation of this progress report. Contributions are grouped according to the simplified CRediT categories of Approach, Technical output, Writing, and Project management.

Table CS1: Declaration of group contributions to the report.

Author	Team Role	Approach	Technical Output	Writing	Project Management

Table CS2: Acknowledgement of AI Tool Use

Group member name	**Specific sub-sections or paragraphs where AI was used**	**AI Tool Use Category/ies**

Table CS3: Group Member Comments

Group member name	**Comments**

By signing below students are acknowledging that the contributions recorded above are a true representation of the contribution made by each individual.

Table CS4: Signature Acknowledgement of Contribution

Name	**Student ID**	**Signature**

4 Methods, Results and Discussion

5 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements

A single python 3 program on the single-board computer (SBC) handled both the flight controller and the SDR in one loop. The code was split by device, a `Drone` class, which handled the flight controller interface, a `Radar` class, which handled the radar interface, and a small `main.py` script to run the overall program. Synchronising the radar captures and the telemetry is critical, as each radar capture needs a position and attitude before it can be georeferenced or used in later processing.

5.1 Introduction

To synchronise flight telemetry and radar measurements, a pilot-initiated control signal had to be interpreted, processed, and transmitted through the entire drone integration system. This required the flight controller to be configured to output the signal, for the signal to be propagated from the transmitter to the radar, and for the subsequent radar capture to be synchronised with the drone telemetry.

First a switch was activated manually on the transmitter, which was then propagated to the flight controller. The flight controller then output this signal over a drone communication protocol called MAVLink to the SBC. The SBC then triggered a capture request to the SDR accordingly. The SDR started the capture and transmitted the information back to the SBC. The signal path is shown in Figure 5.1.

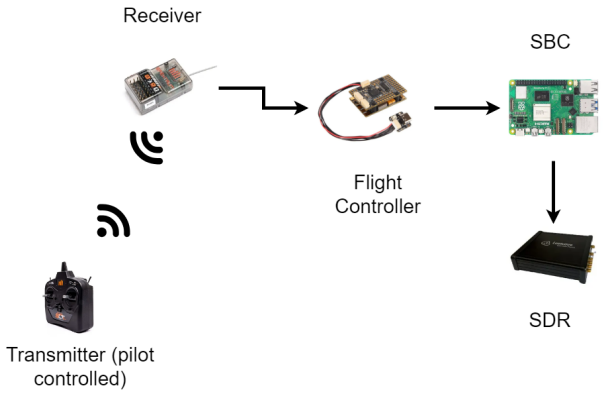


Figure 5.1: Activation signal flow starting at the pilot transmitter and ending at the SDR.

This objective included researching and procuring a suitable SBC, configuring the flight controller, designing software to interface with the flight controller and radar, implementing a post-processing script, and testing the built synchronisation system. The requirements of the system were the ability for a pilot to trigger the radar capture directly from a switch on the transmitter, and that every capture is associated with a telemetry log entry. The final system was a MAVLink-based Python program on board a Raspberry Pi 5 that interfaced with both the radar and the flight controller, synchronising the radar and drone telemetry.

5.2 Flight Controller Configuration

The flight controller needed to be configured to read in a signal from the transmitter and tie that signal to a specific channel. There were 8 physical switches on the transmitter. The switch that was deemed suitable was tied to RC channel 15 using the transmitter settings. The telemetry

log only records global positioning system (GPS) when the drone is armed. This setting did not suit the project, as the drone needed to be disarmed for the majority of the project. To enable GPS logging while disarmed, the `LOG_DISARMED` parameter was changed in the config file to HIGH on the flight controller. Eventually the .tlog file was replaced by a .csv file on board the SBC as it was able to read all relevant telemetry status' directly.

5.3 Final System

The benefit to the program being one system was the fact that all the timing was able to use a single clock, which was `runtime` (seconds elapsed since the start of the program). Both the telemetry log and the radar used this value, and shared the same time base, making radar capture and telemetry synchronisation possible.

To verify the program on the SBC, three tests were conducted, evaluating the key requirements of the system; GPS, radar capture response, and computational overhead. The GPS drift and radar tests were done using existing testing data. This was due to it being infeasible to carry the drone out for another test solely for the purpose of obtaining benchmarks on one subsystem. The computational overhead test was conducted on a separate occasion, as it was possible to do so from indoors.

5.3.1 MAVLink Interface

The SBC interfaced with the drone via MAVLink, a communication protocol used to interface between flight controllers and desktop computers, which the Raspberry Pi 5 running Ubuntu resembles in architecture. The physical connection was over universal serial bus (USB), and the software interface was handled by the `pymavlink` python package. The communication method was originally PWM with one of the servo channels on the flight controller, however this was replaced with MAVLink because MAVLink only required one wire to connect instead of two, and was not dependent on the safety switch of the drone (the first iteration is shown in Appendix **??**).

The program blocked until it received a `HEARTBEAT` signal from the flight controller which confirmed the link and gave the target system and component IDs to be used by later commands. Instead of relying on default stream rates, the program sent `MAV_CMD_SET_MESSAGE_INTERVAL` commands to request each message from the flight controller at a fixed rate. The message IDs of the key messages is shown in Table 5.1.

Table 5.1: MAVLink messages requested from the flight controller and their use.

Message	ID	Rate	Used for
`GPS_RAW_INT`	24	1 Hz	Fix type, satellites, latitude, longitude
`GLOBAL_POSITION_INT`	33	5 Hz	Altitude (AMSL and relative), groundspeed
`ATTITUDE`	30	5 Hz	Roll, pitch, yaw
`BATTERY_STATUS`	147	1 Hz	Voltage, current, remaining capacity
`VFR_HUD`	74	2 Hz	Requested but not parsed
`RC_CHANNELS`	65	5 Hz	Channel 15 (radar trigger)
`HEARTBEAT`	0	Default	Armed state, flight mode

On each loop in the `main.py` file, `update()` drained every queued message with a non-blocking `recv_match(blocking=False)`, and so the program was never stalled by a slow link as the stored value was always the latest received.

Raw MAVLink fields were converted to International System of Units (SI) units upon reception,

including converting latitude and longitude from degE7 to degrees, altitude from millimetres to metres, and voltage from millivolts to volts. The armed status and flight mode of the drone was decoded as part of the `HEARTBEAT` message sent from the flight controller. The radar was triggered from a switch on the pilot's transmitter which was mapped to RC channel 15. The signal was read through the `RC_CHANNELS` MAVLink message (shown in Table 5.1).The trigger logic was rising edge detection, because the encoded data in the MAVLink message was pulse width.

A capture was requested by the main script to the radar only when the pulse width was above 1800 μs. The switch had to be seen as low before a new capture could be requested (which was a pulse width below 1200 μs). The logic prevented one switch from firing repeatedly, ignored the noise in the invalid range, and prevented a switch left high at startup from requesting a capture. Both pulse width parameters can be changed easily in the config file. Buffering was not used, if the radar was still busy, the request was rejected and a message printed, instead of the request being queued. The reason for this was predictability. The timespan of the radar capture is too long for buffering to be feasible.

5.3.2 Test: RC Switch Trigger Latency

An average of the length of time taken for the radar to capture between activating the trigger and starting the capture was taken for all five logs from the tests. Between the first four tests and the last one, the settling time of the radar before capture was changed, which affected the latency. As such in the results, the first four tests the mean latency is subtracted by 4 s to offset the settling time. The results are shown in Table 5.2.

Table 5.2: Trigger-to-capture latency after removing the 4 s settle-delay from logs 1–4

Group	Events	Mean latency (s)	Std. dev. (ms)
Logs 1–4	10	2.0084	0.41
Log 5	3	2.0030	0.24
All	13	2.0071	2.4

Discussion

The mean latency across all tests is approximately 2 s. The low standard deviations of 0.24 ms and 0.41 ms within each configuration shows that the trigger path is stable. The higher standard deviation when all the data is combined is a reflection of the two sets being mixed despite being in different configurations. The program logged data at 1 Hz, so the switch could have flipped up to 1 second extra before the logged row, so the true latency is thus between approximately 2 and 3 seconds, and the numbers shown in Table 5.2 is a lower bound.

The latency would have no effect on synthetic aperture radar (SAR), as SAR requires knowing the position of the antenna during a capture, not how long after a switch a capture started. Also, the GPS is recorded in sync with the SDR, as the program is non-blocking. Whether the latency is acceptable depends on the pilot's user experience, which varies with the pilot. However, during testing the latency was found to be noticeably inconvenient. To improve the latency, the flush and settle time of the radar can be shortened.

5.3.3 Telemetry Log

On each script startup a telemetry log in the form of a .csv file was created from the program, named with the start date and time, so that a telemetry log never overwrote a previous log. Each row in the log held sixteen telemetry fields and five radar fields of data, in addition to the program runtime. Among the five radar fields was the file name of the radar capture (if one occurred), so that each radar capture could be matched directly to the telemetry rows.

To view live data for verification and testing, a terminal dashboard (which was viewed over secure shell (SSH)) showed flight state, GPS, attitude, battery voltage, the pulse width of the RC channel tied to the activation of the SDR, and the radar status. The display was refreshed at one Hz; a parameter changeable in the config file. A screenshot of the display with demo values is shown in Figure 5.2.

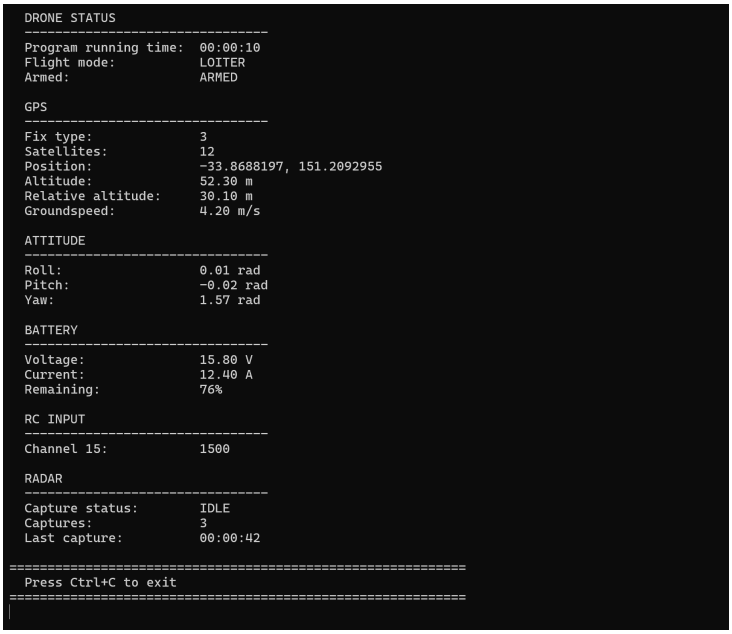


Figure 5.2: Captured screenshot of live data display for the MAVLink interface (Note: data has demo values).

5.3.4 Test: GPS Drift

Windows of time where the drone was stationary during the tests were taken as data samples used to measure stationary GPS drift, where "stationary" was interpreted as having a ground-speed of 0.1m/s. The mean latitude and longitude of the stationary windows are shown in Table 5.3.

Table 5.3: Mean position of each stationary window

Window	Duration (s)	Mean latitude (°)	Mean longitude (°)
Log 1-A	72	−34.9183751	138.6055408
Log 2-A	33	−34.9183822	138.6055543
Log 3-A	16	−34.9183846	138.6055478
Log 3-B	125	−34.9183355	138.6055030

The standard deviation of the distance from the mean, and the maximum distance of the mean

are shown in Table 5.4.

Table 5.4: Horizontal GPS drift and satellites visible for each stationary window

Window	RMS distance from mean (m)	Max. distance from mean (m)	Satellites visible, mean (min–max)
Log 1-A	0.61	1.14	30.2 (27–32)
Log 2-A	0.05	0.11	30.5 (28–32)
Log 3-A	0.07	0.17	31.8 (31–32)
Log 3-B	0.34	0.65	31.0 (23–32)

Discussion

The horizontal standard deviation ranged from 0.05 to 0.61 m, with the worst case maximum distance from the mean being 1.14 m. Log 2-A and Log 3-A had highest stability, with standard deviations of 0.05 m and 0.07 m respectively. The largest drift was shown by Log 1-A, which had standard deviation of 0.61 m. The position moved steadily away from the starting point, reaching 1.82 m after 72 s, despite ground-speed remaining below 0.03 m/s. Log 3-B was in between, with a standard deviation of 0.34 m. There is a large dip in the satellite count for log 3-B. It can be seen that Log 1-A has the same approximate satellite count as Log 2-A, yet has a standard deviation of 0.61 m against 0.05 m. The results indicate that the difference in drift does not correspond to the satellite count alone. The two shortest windows had the lowest GPS drift, so it is evident that drift may grow with window length. However, drift is not necessarily directly proportional to duration, as the highest drift was recorded by Log 1-A, which has a shorter duration than Log 3-B.

For georeferencing, the GPS drift does not interfere with the ability to plot where each capture was taken. The location where the data were taken was between two large buildings, which likely negatively affected the GPS drift. Because of the location, a level of inaccuracy was expected. Operational steps to reduce GPS drift range from letting the GPS warm for a number of minutes before operating, and distancing from buildings and metal to reduce multipath.

5.3.5 SDR Interface

The SBC and radar did not communicate directly. Instead, the capture executable file was run by the `Radar` class in the program as a subprocess. This was to ensure appropriate separation between the two systems in order to minimise complexity. Under this method, the SBC was only responsible for scheduling and bookkeeping of the radar captures.

The capture settings for the radar existed in a JavaScript Object Notation (JSON) file which was read by the program at startup, so that parameters could change between flights without editing code. Among these settings were the executable and firmware paths, output folder for the radar captures, duration of capture, and sweep time. The radar capture cycle was modelled as a state machine, the diagram is shown in Figure **??**.

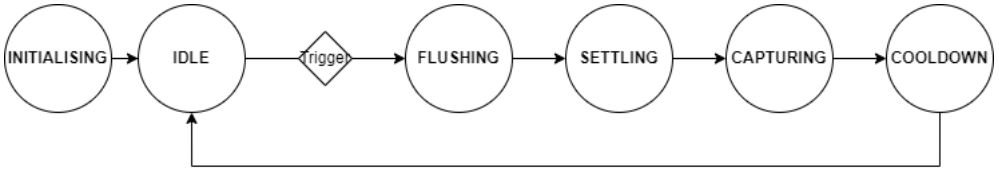


Figure 5.3: State diagram of radar control sequence

Each capture file was named according to its capture number and runtime on capture. Together with the per-row radar columns in the CSV file, this gave two independent ways to align the radar capture data with the telemetry data sets.

5.3.6 Test: Computational Overhead

To address the tight computational requirements of the airborne system, a computational overhead test was conducted. Three seperate tests were run. The first test was recorded while the system was sitting idle and not running the program. The second test was recorded during program runtime and no radar captures were made. The third test was recorded during program runtime while continously capturing radar data. The first test provides a baseline for the system usage, the second a baseline for the program, and the third the behaviour of the system during operation. The results for the three tests are shown in Table 5.5

Table 5.5: Raspberry Pi resource usage for each test condition

Condition	Samples	Duration (s)	System CPU (%)	Script CPU (%)	RAM used (%)	Temp. (°C)
Baseline (script off)	313	313.5	0.07	–	3.60	48.17
Script running	163	163.3	0.12	0.16	3.77	47.00
Script running, capturing	171	171.3	0.30	0.17	3.85	47.57

Discussion

Each test made a continual recording of the percentage CPU used by the system, percentage CPU used by the program, percentage RAM used by the system, and the temperature of the system. The system CPU usage increases steadily across the three tests, but remains at less than 1% usage. The script central processing unit (CPU) usage is higher than the system CPU usage on one occasion. This is not an error, and is due to the system CPU usage being averaged over all four CPU cores, while the script is measured against one core. The RAM usage increases from 3.6% to 3.85%, and the average temperature remains approximately constant. From this it can be concluded that the program utilises a substantially small amount of computational resources, and so is enough leftover computational resources for performing live radar data processing on the Raspberry Pi 5 in a future design iteration.

5.3.7 Post-Processing

A post-processing script that takes the synced .csv file from the SBC and draws the flight path on an interactive map was created. The script is shown in Appendix A.5. When a new radar capture starts, a marker is placed on the interactive flight map at the location. The script erases the first row of data, as the longitude and latitude start at invalid value 0. This is because the flight controller initially outputs default values before beginning data recording. Coordinates are already in decimal degrees from the SBC program, so they are plotted directly. The script was run on a desktop personal computer (PC), not the SBC. Data visualisation was done after capturing, so using a PC was appropriate. SDR data was not processed in this stage. An example output is shown in Figure 5.4, which was from testing the RC switch latency.



Figure 5.4: Flight path and radar capture locations plotted in the data visualisation script.

The blue line is a point-to-point line between GPS data points, and the markers indicate points where radar data was captured. The markers also contain information about each radar capture (row, capture count), and there is one marker per radar capture.

5.4 Summary

Objective 4 implemented the full control cycle of the SDR integration system, enabling a switch on the transmitter to trigger radar captures, and for those captures to be synchronised to drone telemetry. A Raspberry Pi 5 was chosen as the centralised control system as it leaves room for future onboard processing. The final version of the program used MAVLink over USB, which simplified wiring and removed the dependency on the drone's safety switch. The program logged telemetry and radar state to one .csv file per run. A latched trigger ran the radar capture sequence. Afterward, a seperate program on a PC plotted capture locations on a Hypertext Markup Language (HTML) map.

When the system was stationary, the GPS position drifted 0.05-0.61 metres from the mean, with a worst case drift of 1.14 m. The average trigger-to-capture latency was 2.007 seconds over 13 recorded captures. Due to the system's refresh rate, the actual latency is 2-3 seconds, which can be adjusted. The computational overhead of the program was negligible; the CPU usage remained below 1% and the random-access memory (RAM) usage rose only from 3.6% to 3.85% in the most computationally demanding tests. The primary limitations were a settle-delay setting error in the first four logs and the reuse of earlier testing data, instead of staging new tests. Future work is extending the program to include on board processing (utilising the Raspberry Pi 5's spare processing power), reducing the capture latency by optimising the radar's capture cycle, and conducting further testing to verify the GPS drift.

Statement of AI Use

Spelling and Grammar

The project team used ChatGPT (https://chat.openai.com/) and Claude (https://claude.ai/) for this Progress Report to check spelling and grammar in selected text drafted by the team. The use of AI tool was limited to identifying spelling and grammar errors. The tools were not used to draft new report sections, produce summaries or paraphrases, restructure the report, interpret results, draw conclusions. The project team reviewed all suggestions before making any changes and retained copies of the original text submitted to the AI tools.

The following prompt was used in May 2026:

> Please review the following paragraph and identify spelling and grammar errors only. Do not rewrite the text, change the meaning, improve the flow, shorten sentences, add content, or make style or substantive editing suggestions.

Initials of group members who used this support: HL, LT, KM, AS.

Research Technical Support

The project team used Claude Code to analyse the legacy MATLAB driver, implement radar commands, and produce code documentation. All code and documentation generated by AI underwent human review and approval. AI use in this form was approved verbally by Dr Brian Ng. The following prompt was used in April 2026:

> Document all functions in the following file using Doxygen-style comments:
>
> `C:\Users\eliza\SDR_main\SDR_main\linux\src\UsbDevice.cpp`
>
> Documentation requirements:
>
> - Add a Doxygen comment block above every function definition.
> - Include:
>   - `@brief`
>   - `@param` for all parameters
>   - `@return` where applicable
>   - `@note` or `@warning` if relevant
> - Keep descriptions concise but technically clear.

Initials of group members who used this support: HL

6 System Design

The purpose of the system design was obtaining a high level idea of how each subsystem interacts with the other, and an understanding on how each subsystem meets the requirements. This chapter covers the system architecture, showing the key subsystems and how they interact. Then, the subsystem interfaces are covered in more detail.

6.1 System Architecture

A diagram of the system architecture is shown in Figure 6.1. The system shows the key subsystems and their interactions, and whether they are located in the drone, the 3D printed compartment, or with the pilot. The activation signal propagates throughout the diagram, starting at the transmitter and ending at the radar. The program on the SBC launches an executable, which provides the radar with the information required to complete the capture. Both radar capture and drone telemetry are synchronised by the SBC. The entire system is powered by the drone's supply, and the system resides in the 3D-printed compartment, beneath the drone.

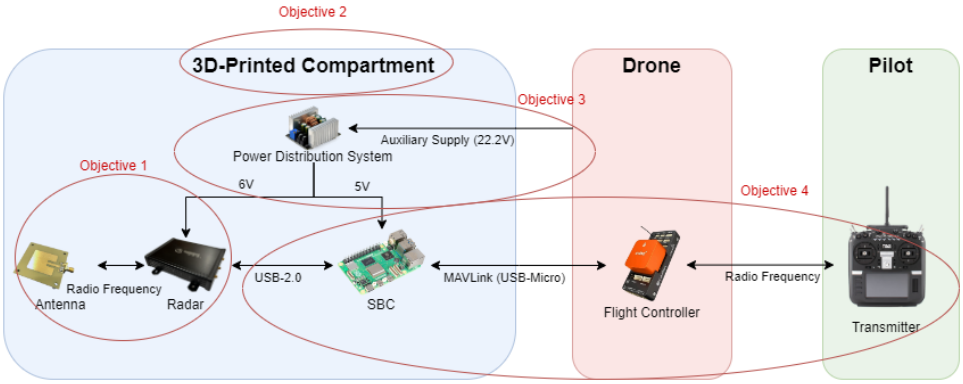


Figure 6.1: High level diagram of drone-radar integration system architecture

6.2 Subsystem Interfaces

Each subsystem interface is listed in Table 6.1 and numbered I1-I8. The interfaces I6 and I7 share the same physical connection and are read by a single program on one clock.

Table 6.1: Interfaces between subsystems

ID	From	To	Data / signal	Link, protocol, rate	Objective
I1	SDR	Antennas	RF transmit / receive	Coaxial	1
I2	Pi 5	SDR	Capture control, IQ samples	USB 2.0 high-speed	1, 4
I3	Auxiliary Supply	Power Module	Power (22.2V)	XT60	3
I4	Power Module	Pi 5	Power (5V)	USB-C	3
I5	Power Module	SDR	Power (6V)	Barrel-Jack	3
I6	Flight Controller	Pi 5	GPS position, fix, satellites	USB-Micro, MAVLink, 1 Hz	4
I7	Flight Controller	Pi 5	RC capture switch state	USB-Micro, MAVLink, 5 Hz	4
I8	Flight controller	Transmitter	Telemetry, commands	radio, MAVLink	4

A Objective 4: Supporting Material

A.1 SSH Setup

Listing A.1: The commands used to setup SSH connection between the Raspberry Pi and a remote computer.

```
sudo apt install openssh-server
sudo systemctl enable ssh
sudo ufw allow ssh
```

A.2 GPIO Packages Install

Listing A.2: Bash command used to install python general purpose input/output (GPIO) library.

```
sudo apt install python3-lgpio gpiod
```

A.3 Virtual Environment and `rpi-lgpio` Install

Listing A.3: Commands used to create the python virtual environment and install `rpi-lgpio`.

```
wget http://abyz.me.uk/lg/lg.zip
unzip lg.zip
cd lg
make
sudo make install
python3 -m venv gpio-env
source gpio-env/bin/activate
pip install rpi-lgpio
```

A.4 GPIO Read Script

Listing A.4: Python script used to run on the Raspberry Pi to monitor one of the GPIO pins.

```
#!/usr/bin/env python3
import RPi.GPIO as GPIO # Uses rpi-lgpio backend automatically
from time import sleep
import signal

# Setup
GPIO_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(GPIO_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

print("Sensing 3.3V High...")

try:
    while True:
        if GPIO.input(GPIO_PIN):
            print("Pin High")
            sleep(1)
except KeyboardInterrupt:
    print("Stopping...")
```

```
finally:
    GPIO.cleanup()
pause()
```

A.5 Telemetry Data Visualisation Script

Listing A.5: Python script used to plot and visualise syncrhonised radar and telemetry data

```
import folium
import pandas as pd
import os

# parameters
filename = "flight_log_5.csv"


# Code
df = pd.read_csv(filename)

# first lat/lon sample is invalid, drop it and reindex from 0
df = df.iloc[1:].reset_index(drop=True)

output_file = os.path.splitext(filename)[0] + ".html" # -> "flight_log_4.html"
os.remove("synced_data.html") if os.path.exists("synced_data.html") else None

m = folium.Map(location=[df.at[0, "latitude"], df.at[0, "longitude"]], zoom_start=100)

# Create list of [latitude, longitude] points
points = list(zip(df["latitude"], df["longitude"]))

# Draw flight path
folium.PolyLine(points, weight=3).add_to(m)

capture_count = 0
for i, row in df.iterrows():
    tooltip_html = (
        f"<b>#{i}</b><br>"
        f"Latitude: {row['latitude']:.6f}<br>"
        f"Longitude: {row['longitude']:.6f}<br>"
        f"Capture Count: {row['radar_capture_count']}<br>"
    )

    if row["radar_capture_count"] == capture_count + 1:
        folium.CircleMarker(
            location=[row["latitude"], row["longitude"]],
            radius=3,
            color="blue",
            fill=True,
            fill_opacity=0.8,
            tooltip=tooltip_html,
        ).add_to(m)
        capture_count = 1 + capture_count
# Save map
```

```
46 m.save(output_file)
```